

Proyecto Escape the Maze

Realizado por: Raúl Silva Bienvenido y Jose Carlos Leal Iglesias

Índice

Introducción.....	3
Objetivos.....	3
Alcance.....	3
Memoria.....	4
Descripción.....	4
Módulo juego.....	4
Módulo periféricos.....	5
Diagramas y Esquemas.....	6
Referencias.....	9
Conclusiones.....	10
Dificultades.....	10
Mejoras.....	10
Guía de uso.....	11

Introducción

Objetivos

El objetivo principal de este proyecto es desarrollar un juego sobre escapar de un laberinto para un microcontrolador LPC 4088. En este juego la iluminación del personaje está controlada por un sensor de luz de tal manera que hasta que no se ilumine lo suficiente este sensor, la pantalla permanecerá en negro.

Como objetivos más concretos, están: el manejo de la pantalla LCD de manera correcta para mostrar los píxeles visibles por el jugador en cada momento, junto a las conexiones del sensor de luz y de un joystick externo adecuadamente, y por lo tanto, la utilización de los valores de estos periféricos para conseguir la iluminación requerida y el movimiento del personaje respectivamente.

Alcance

El proyecto parte de un kit de desarrollo que contiene microcontrolador LPC 4088 y una pantalla LCD ya conectada a la placa junto a los ya mencionados periféricos, el sensor de luz y un joystick. Además, se dispone de todas las funciones creadas en las prácticas de esta asignatura. En base a estos materiales, se alcanza el objetivo final, generando un laberinto donde la movilidad del jugador es controlada por el joystick, y la visibilidad por el sensor de luz, obteniendo así un pequeño juego.

Memoria

Descripción

El desarrollo del proyecto se ha realizado siguiendo una filosofía modular, cuya estructura se divide en dos grandes módulos: la lógica del juego (juego-laberinto, desarrollado principalmente por Raúl Silva Bienvenido) y la conversión de los valores analógicos a digitales (adc_periféricos, desarrollado principalmente por Jose Carlos Leal Iglesias). Estos dos módulos nos han permitido poder enfocarnos con mayor facilidad en el proyecto, haciendo más eficiente el desarrollo y evitando bloqueos gracias al carácter paralelo de estos.

Módulo juego

En cuanto al juego, este consiste en el uso de dos matrices principales:

- Paredes del laberinto. Esta matriz es de las dimensiones de la pantalla LCD dividido por 10, permitiendo ahorrar espacio en memoria y mantener el laberinto en todo momento. Por lo tanto, cada pared del laberinto representa 10 píxeles en la pantalla. Gracias a eso, conseguimos tener una matriz con el laberinto completo, mientras que ganamos eficiencia y espacio en la memoria del microcontrolador.
- Visión del jugador. Esta matriz es la más importante, pues es la que se utilizará para mostrar por pantalla todo lo que vea el jugador. Esta matriz, a diferencia de la referente a las paredes, si es de su tamaño original, esto es debido a que su tamaño, comparado con las dimensiones de la pantalla, son mucho más pequeñas. En esta matriz, los valores que contiene son referentes al jugador y a su cono de visión, con una forma parecida a un triángulo invertido.

Con esto, la función más importante y de mayor peso del módulo de juego es 'dibujar_cono_vision', la cual como su nombre indica, es la encargada de realizar los cálculos necesarios para que la visión del jugador se refleje por la pantalla LCD. Su importancia se debe mayormente a que, realiza el escalado de las paredes del laberinto para que se vea como era intencionado (es decir, la conversión a los 10 píxeles) y los casos de rotación en el que el jugador cambie de dirección. Esto es lo más crítico, ya que únicamente tenemos una matriz de visión con una dirección estática, por lo que según la dirección que tome el jugador con el joystick se debe calcular la posición real en la que se mostrará en la pantalla.

Por lo demás, el resto de funciones son auxiliares para facilitar el cálculo anterior, excluyendo obviamente a la función inicializadora, cuyo objetivo es inicializar todas las dependencias del módulo, y la función 'comenzar_juego', la cual muestra el menú principal y actúa de "centro de control" frente a las decisiones tomadas por el jugador.

Módulo periféricos

En este módulo se describen todas las funciones necesarias para la lectura de los valores de los 2 periféricos, así como macros y constantes para hacer más legible y modificable el código. Consta principalmente de 2 funciones y una función más para inicializar todos los pines del microcontrolador:

- La primera función es 'inicializar_periféricos', como se ha mencionado antes se encarga de inicializar los canales del adc a los que se han conectado los periféricos (Canal 5 para la LDR, canal 0 para el eje x y canal 1 para el eje y del joystick) usando una frecuencia pasada como parámetro. Además, esta función incluye la configuración del pin 21 del puerto 2 como puerto de entrada con resistencia de pull-up para poder filtrar correctamente las pulsaciones del botón que incluye el joystick.
- La segunda función que se implementa es 'encender_linterna', esta lee el valor del sensor de luz haciendo uso de la función ya creada en las clases 'adc_convertir' y compara este valor con un valor predefinido en la macro UMBRAL_LUZ de tal manera que si es mayor que este valor devuelve OSCURO y si es menor o igual devolverá ILUMINADO (esto se debe a que cuanto más luz reciba la LDR menor será el valor de voltaje que deja pasar, a mayor luz, menor valor en el adc). OSCURO e ILUMINADO son dos valores de un tipo *enum* creado para que sea más entendible y fácil de usar en el módulo de juego.
- La tercera función desarrollada es 'joystick_leer', esta función se implementa de manera muy similar a la usada en la primera práctica para leer el joystick de la placa, con la única diferencia de que para devolver los valores de arriba, izquierda, derecha y abajo se utiliza la función 'adc_convertir' para leer los valores de los canales del adc a los que están conectados el eje 'x' y el eje 'y' del joystick, y, nuevamente, se comparan con 2 valores predefinidos (UMBRAL_ABAJO_DERECHA y UMBRAL_ARRIBA_IZQUIERDA) en macros para devolver cada valor del tipo *enum* como se hacía en la función ya mencionada. También esta función lee el pin configurado con la resistencia de pull-up al que está conectado el botón del joystick.

Diagramas y Esquemas

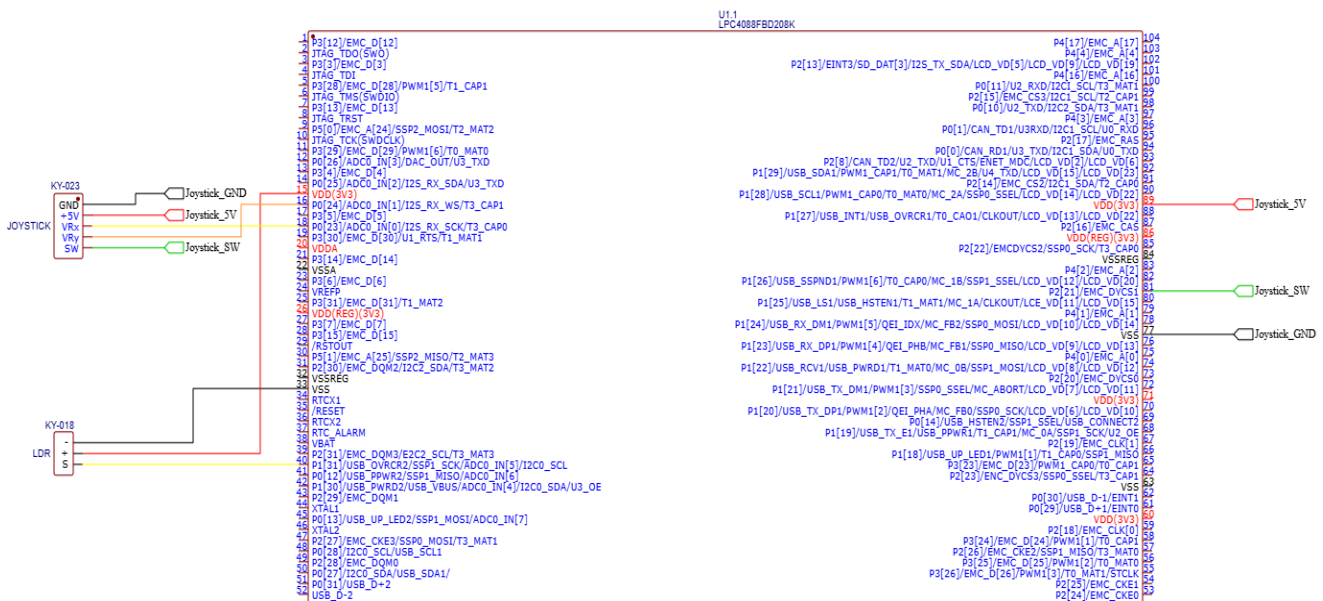


Diagrama físico del sistema creado mediante (EasyEDA, n.d.).



Diagrama funcional del sistema.

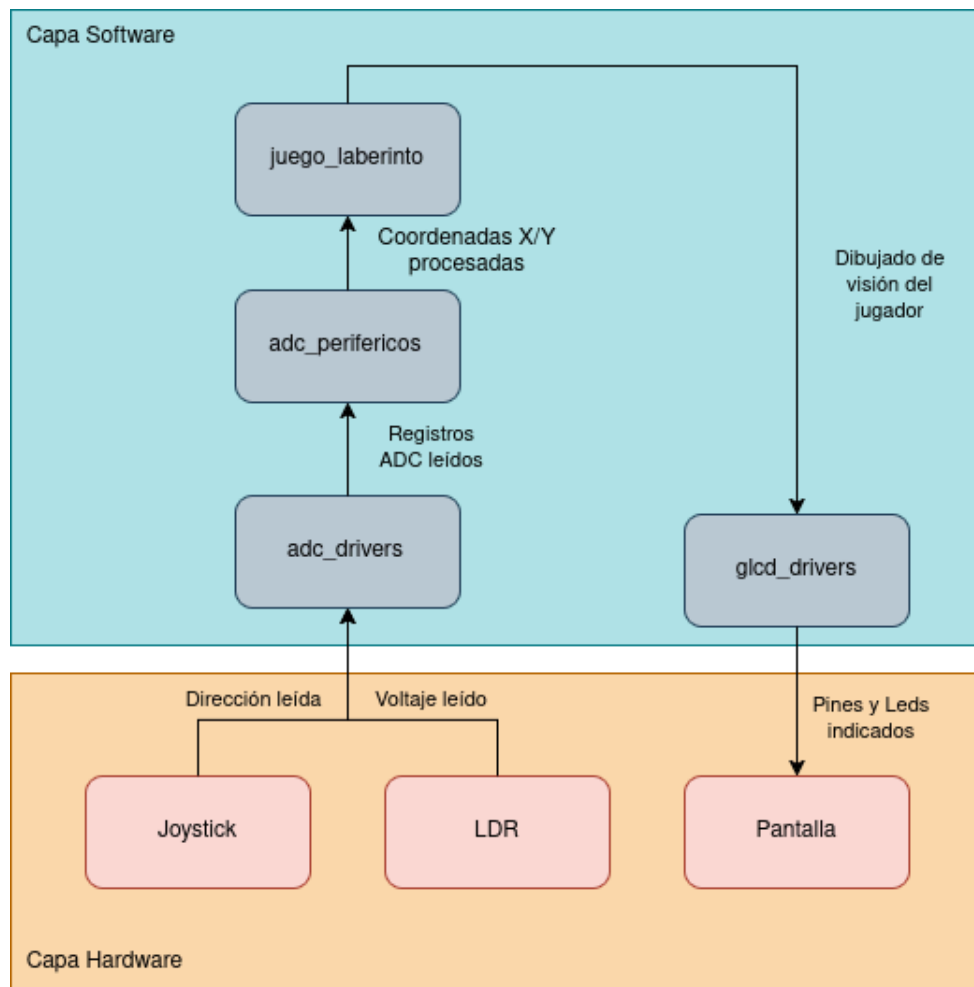


Diagrama de bloques de organización del programa.

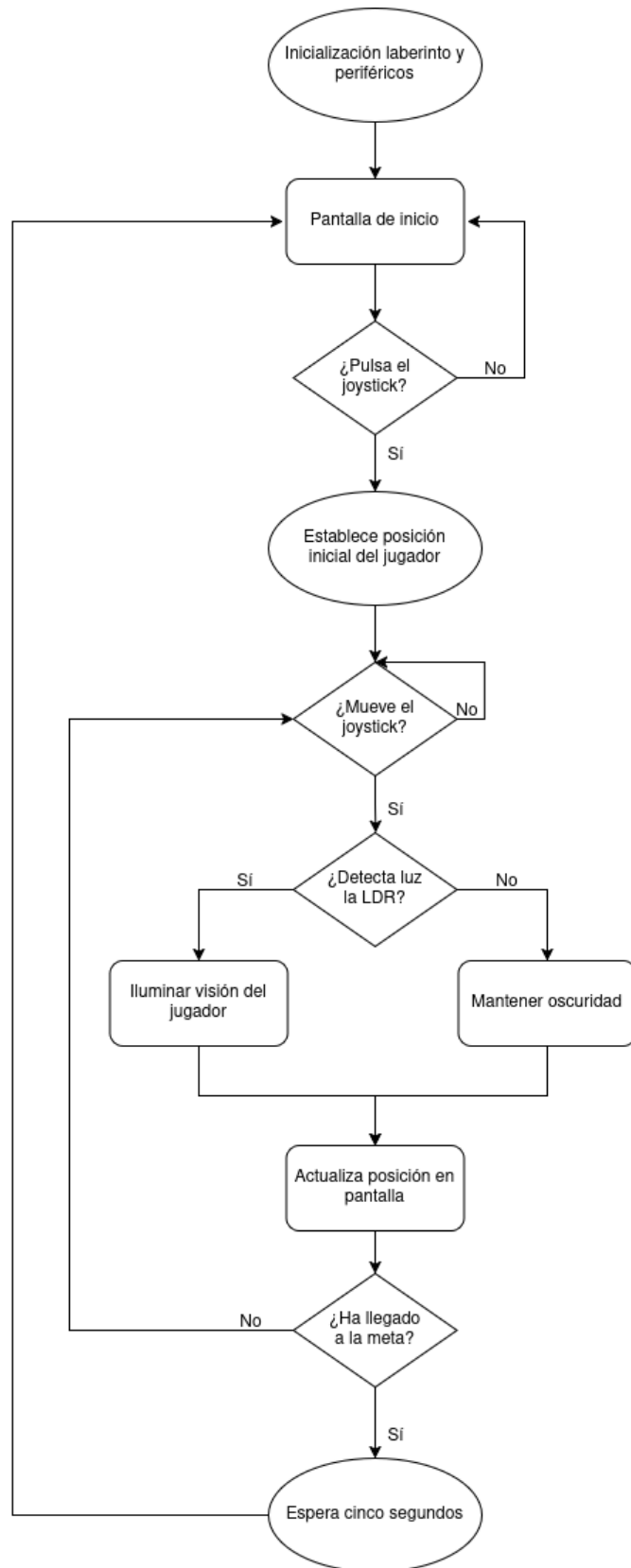


Diagrama de bloques de funcionamiento del programa.

Referencias

EasyEDA. (s.f.). *EasyEDA - Online PCB design & circuit simulator*. Accedido el 7 de junio de 2026, de <https://easyeda.com/>

GitHub. (s.f.). *GitHub: Where the world builds software*. Accedido el 30 de abril de 2026, de <https://github.com>

Google. (2026). *Gemini 3.5 Pro* (Versión de junio de 2026) [Gran modelo de lenguaje]. Accedido el 28 de mayo de 2026, de <https://gemini.google.com>

NXP Semiconductors. (2024, 12 de marzo). *User Manual (UM10562)* (3.^a ed.). Utilizado el 21 de mayo de 2026.

Conclusiones

Dificultades

Respecto al módulo de juego, las mayores dificultades han sido respecto a la pantalla LCD y a la visión del jugador. Siendo el problema de la LCD es sobre todo sus dimensiones, las cuales limitaban el hecho de generar matrices masivas. En cuanto a la visión del jugador, la mayor dificultad ha sido desarrollar la rotación del cono de visión, siendo un desafío el generar el cálculo necesario para mostrar su cono de visión según la dirección del jugador.

Por parte del módulo de periféricos, la gran dificultad ha sido, sin duda, el manejo de los 3 canales del adc mediante interrupciones llegando hasta el punto de no incluirlo en el proyecto al no conseguir que funcionara. Se intentó tanto lanzar todas las lecturas una a una como usar el modo burst del adc y de ninguno de estos 2 métodos se conseguían lecturas del adc.

Mejoras

Como mejora clave queda la implementación por interrupción de las lecturas de los periféricos, además, como mejora del juego en sí, se pueden añadir más laberintos y que el cono de visión no solo esté encendido o apagado sino que pueda haber una transición progresiva de un estado a otro lo que permitiría al jugador a poder jugar con diferentes grados de luminosidad.

Guía de uso

En este apartado se redactan los pasos a seguir para poder ejecutar el programa en la placa LPC4088.

- 1. Conectar periféricos:** Lo primero que se debe hacer es conectar nuestros 2 periféricos (LDR y Joystick) a la placa apagada. Conectaremos los 3 pines del sensor de luz de tal manera que el pin donde está el símbolo menos en el sensor lo conectamos al pin 61 o 63 del conector J3 de la placa, el pin intermedio que sería el de corriente lo conectamos al pin 62 o 64 y, por último, el de la señal analógica (S en el sensor) lo conectamos al pin 39 (que corresponde al canal 5 del adc de la placa). Para conectar el joystick, usaremos el conector J15, conectaremos el pin GND y +5V del joystick a los pines 1 y 2 respectivamente. El pin de VRx se conecta al pin 13 del conector, el pin VRy al pin 14 del J15 y, por último, el pin SW lo conectamos al pin 12 de este conector que corresponde al pin 21 del puerto 2 del microcontrolador.
- 2. Abrir y cargar el proyecto:** Abrimos el archivo de proyecto desde Keil μ Vision en project/proyecto.uvprojx. Una vez abierto, pulsamos Ctrl + F7 (Translate) y F7 para compilar y generar el ejecutable. Conectar la placa mediante J-Link y pulsar F8 (Download) para cargar el programa en la memoria Flash del microcontrolador.
- 3. Disfrutar del juego:** Si todo ha ido bien, sólo queda pulsar el botón del joystick para salir de la pantalla de inicio, una vez en el juego, ilumina la LDR hasta que se ilumine el cono de visión y usa el joystick para escapar del laberinto.